

Projeto 1 (Complexidade Computacional)

Angel Jhesua Machado Rodriguez

March 26, 2026

A partir da realização dos itens, A, B, C e D, usando matrizes $A_{n \times k}$ e $B_{k \times m}$ com entradas aleatórias da distribuição normal, obtive os seguintes resultados:

Caso	m, k, n	mat_prod	mat_prod_dot	matmul
$m < n$,	(15, 10, 30)	0.003127781	0.000690866	0.0000352
$m > n$,	(30, 10, 15)	0.003209223	0.000656579	0.0000217
$m = n$,	(30, 30, 30)	0.020553339	0.001302895	0.0000373
$m < n$,	(300, 200, 500)	20.19664878	0.256992501	0.0104510
$m > n$,	(500, 200, 300)	19.01495256	0.256424507	0.0017140
$m = n$,	(500, 500, 500)	79.66718074	0.600667979	0.0069889

Table 1: Tempo em segundos, 1 execução - Python

mat_prod	mat_prod_dot	matmul
1	4.52	88.77
1	4.89	148.07
1	15.77	550.30
1	78.58	1932.50
1	74.15	11071.90
1	132.63	11398.99

Table 2: Desempenho relativo a mat_prod - Python

Caso	m, k, n	mat_prod	mat_prod_dot	matmul
$m < n$,	(15, 10, 30)	0.0000081	0.0000539	0.00000333
$m > n$,	(30, 10, 15)	0.0000120	0.0000424	0.00000302
$m = n$,	(30, 30, 30)	0.0000256	0.0001345	0.00000810
$m < n$,	(300, 200, 500)	0.0116126	0.1057130	0.00123698
$m > n$,	(500, 200, 300)	0.0107628	0.1033686	0.00243282
$m = n$,	(500, 500, 500)	0.0433801	0.7200440	0.00516706

Table 3: Tempo médio em segundos, 10 execuções - Julia

mat_prod	mat_prod_dot	matmul
1	0.15	2.43
1	0.28	3.98
1	0.19	3.16
1	0.10	9.38
1	0.10	4.42
1	0.06	8.39

Table 4: Desempenho relativo a mat_prod - Julia

Estes resultados serão utilizados para responder as perguntas específicas de cada item.

Python

Em Python consegui realizar apenas 1 execução para cada caso de teste, antes de cada execução de teste houve uma pré execução de "aquecimento". Como o tempo de cada teste depende da performance que cada cpu pode proporcionar também foram criadas tabelas auxiliares (Tabela 2) contendo o desempenho relativo a função de pior implementação mat_prod

Item B

Podemos ver e comparar o resultado de cada caso de teste olhando para a coluna mat_prod da tabela 1, no casos de teste $m < n$ e $m > n$ apenas troquei o valor de m com o valor de n para saber como isso afetaria o tempo, e podemos ver um tempo levemente maior para valores de m maiores tanto no produto de matrizes menores quanto no produto de matrizes maiores onde é mais notório, por ter realizado apenas uma execução não temos provas empíricas fortes mas serve para suspeitar que a "orientação" das matrizes dependendo da implementação pode mudar o tempo de execução mesmo que a quantidade de operações seja a mesma

Também podemos perceber que o tempo de execução escala bem rápido, no caso (30, 30, 30) o tempo de execução foi relativamente baixo porem quando olhamos o caso maior (500, 500, 500) temos um tempo de 80 segundos para realizar 1 uma única multiplicação

Item C

Ao implementar a função de multiplicação de matrizes usando o produto escalar e aplicando os mesmo testes, podemos ver na tabela 2 na coluna mat_prod_dot que desde as matrizes menores a nova implementação foi aproximadamente 4 vezes mais rápida e a medida de escala o tamanho das matrizes o desempenho relativo cresce de forma moderada

Outra informação interessante é que utilizando o produto interno, a implementação é ligeiramente mais lenta para valores de n maiores que os de m como podemos ver na tabela 1

Item D

Finalmente ao comparar as funções implementadas com a função `matmul` de `numpy` percebemos a abismal diferença de desempenho, desde um início sendo 88 vezes melhor do que a implementação mais simples e a medida que escala o tamanho das matrizes o desempenho relativo cresce abundantemente.

Julia

Graças a que as matrizes são uma estrutura de dados nativa de Julia, até a implementação mais simples teve um desempenho muito bom, a causa disso cada caso de teste foi executado 10 vezes e do tempo de cada execução foi tirada a média

Item B

Diferentemente de Python, em Julia a função `mat_prod` não escala tão rápido como, em Python do caso (30, 30, 30) para o caso (500, 500, 500) temos uma razão de 4000 enquanto em Julia essa razão de 2000, ainda percebemos que um dos casos $m > n$ ou $m < n$ é favorecido mesmo que o número de operações seja o mesmo.

Item C

Surpreendentemente implementar o produto de matrizes com o produto interno de Julia ao invés de melhorar o desempenho, ele cai, podemos ver isso na tabela 3 e mais facilmente na tabela 4, onde o desempenho relativo é menor do que a função `mat_prod`

Item D

A função nativa de produto de matrizes em Julia melhora um pouco o desempenho porém a forma em que ela escala talvez seja um pouco baixo comparado com Python, quando olhamos as tabelas 2 e 4, então realizar um `triple for` para multiplicar matrizes em Julia não é tão pesado quanto em Python